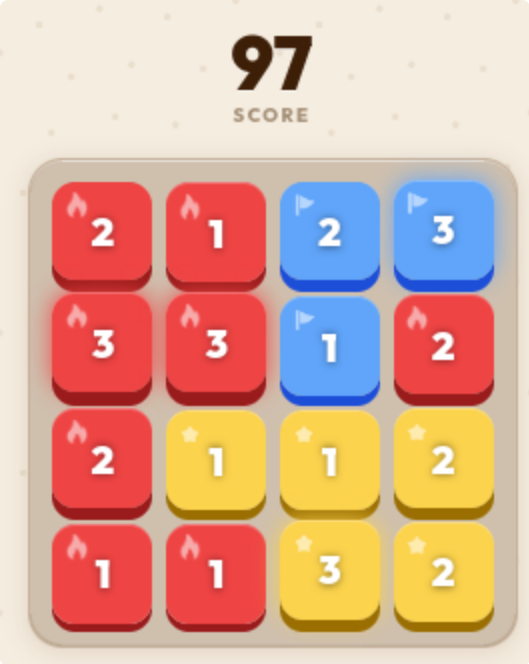
Esta es una documentacion para poder documentar y entender como es que funciona

Primero que todo, que se solicita de la especificacion:

El grupo elige el algoritmo, que puede ser informado o no informado. El README documenta la formulación completa, es decir, la representación del estado, el operador de sucesión, el costo de cada acción, la prueba de meta y, cuando el algoritmo sea informado, la heurística junto con el argumento de su admisibilidad o de la razón por la cual se renuncia a ella. Una heurística no admisible es aceptable si el grupo la declara y explica qué garantía pierde.

Entonces, el documento va a explicar la formulacion para como llegar a esto;

Primero hay que ver como funciona TileUp; para eso se hace lo mas directo;



Jugarlo(mucha habilidad no hay)

De esto se pueden sacar ciertas conclusiones

1. Es determinista(Es decir que en este caso si se pone una ficha en un lugar siempre se dara lo mismo)
2. Es completamente observable, porque el tablero se puede ver en todo momento, y en este caso ve el orden de las fichas que se le da
3. Mientras que en el que jugue es infinito(https://www.reddit.com/r/TileUp/comments/1x0b35m/tile_up_endless_mode_october_7/) hay otros en los cuales es finito y en este caso el motor del juego tiene un fin

4. Debido a la naturaleza del juego, tiene un branching muy alto, ya que como(en teoria segun una estrategia optima) se pueden poner piezas juntas para fusionarlas, se pueden poner en lados aleatorios, o pueden estar en el mismo lado pero con valores distintos

Ya se tienen algunos temas, ahora hay que definir algunas cosas para el algoritmo y poder montar la heuristica

La solucion tiene que ser algo rapida y barata, es decir que no consuma mucho y que sea rapida en resolver ya que toma tiempo

Con esto definido hay que ver que tipo de algoritmo se puede usar.

Se tienen algunas opciones, DFS, BFS que son como las mas basicas pero hay otras ideas como Greedy, Beam, A* que son algunas que se han visto en clase, pero aca hay algunos temas

Primero que todo, como hay tantos caminos posibles, DFS y BFS son extremadamente ineficientes entonces lo mas facil es descartarlos, por lo que se queda con Beam, Greedy, y A*



(remover antes de enviar)

Escoger el algoritmo

Ahora, hay unos temas entre cada algoritmo de busqueda

Primero el greedy, este es un algoritmo informado a corto plazo, ya que solo ve los nodos que tiene frente a el y sigue el mas barato. Se asemeja a DFS en que solo profundiza una unica ruta.

Beam es otra opcion pero si consume mas y es un poco mas dificil de implementar consume mas memoria porque explora mas rutas, pero si hace varias ramas puede ganar bien eso si con este y greedy es un poco dificil la heuristica y no garantiza eficiencia ni completar el programa(aunque al ser tan corto no deberia tener problema).

Finalmente A*
Este es el mas complejo y que tiene mas requerimientos, requiere una heuristica admisible, pero si es asi puede garantizar que complete y que sea optimo, no solo eso, como almacena los caminos no utiles tambien es mas pesado.

Formular la heuristica

Con todo esto, beam es el que se escogio. Entonces, primero, se tiene una cantidad de estados.
Los campos que estan siendo ocupados: T(s)
Los colores que tienen al menos uno en el tablero: Vivos
Los espacios vacios: σ
El color: c

Entonces una posible formula o forma de calcular seria ver la cantidad de todos los valores

T(s) - Vivos * σ * c

Ahora, esto es MUY basico, asi que se puede mejorar, primero, se puede usar el valor beta, que es un parametro ajustable con el beam search, luego, como se ven los colores e incluso el ver las piezas por poner, siempre mirando lo que falta por poner, entonces

$T(s) - \beta * \sum_{vivos} \min(\sigma_c(s) , frecuencia(c, i))$

Representacion de estado

El estado lo maneja mas el motor y se lo pasa al agente

```
Ficha = Tuple[int, int]  # (color, valor)
Tablero = Tuple[Tuple[Optional[Ficha], ...], ...]
```

Ahora, el estado del tablero en si actual se puede ver en filas y columnas, y cada ficha tiene 2 valores, para el color y numero

Como se puede ver, ambas son tuplas, una siendo una tupla de los 2 valores y la otra siendo una tupla de tuplas.

Operador de sucesion

Este tambien lo maneja el juego en si y se lo pasa a el algoritmo de busqueda, lo que hace es que revisa las acciones que sean legales, entonces lo que hace es que coloca la ficha, y si puede hacer una fusion la hace.

Entonces

Primero cuando se coloca la ficha, se ve el color y se revisa las fichas adyacentes y si es que siquiera hay. Entonces, si es que hay 2 o mas fichas adyacentes que son del mismo color y valor, se fusionan y se queda esta unica.

Costo de acciones

El costo de cada accion es 0. Esto se debe a que la cantidad de fichas o pasos se define al inicio, sin importar que, esto es porque sin importar que se van a tener que poner la misma cantidad de fichas.

Prueba de meta

```
 if es_meta(estado, instancia):
     return False
 return len(celdas_vacias(estado)) == 0
```

```
es_meta(estado: Estado, instancia: Instancia) -> bool:
"""Victoria: se colocaron las M fichas de la secuencia."""
return estado.indice_siguiente >= instancia.m
```

Aca es donde se verifica la condicion de meta, entonces esto va de la siguiente forma

La condicion de meta revisa si ya se colocaron todas las fichas, si es asi, listo, se termina.

La de derrota es un poco mas compleja, ya que primero revisa si se termino, si no se ha terminado y ya no hay espacio, pues en este caso ahi es donde se llega la derrota